

ECAS PHP Client 2.x

Support will be provided at best effort. Use the centralised FMB for questions on the SDK

Table of Contents

- Table of Contents
- Version history
 - Version 2.2.x
 - Version 2.1
 - Version 2.0
 - Composer
- Installation
 - Composer
 - Check Composer connection
 - Install the dependencies
- How does the PHP Client work?
- Configuration
 - Certificates
- CacheEngine
 - NullCache
 - About NullCache
 - ApcCache
 - About APC
 - Enable cache engine in properties
 - APCu Cache
 - About APCu
 - Enable cache engine in properties
 - Memcached
 - About Memcached
 - Enabled cache engine in properties
- Logging
 - Configuration
 - How to use
- Interceptors
 - Interface
 - Implementation example

Version history

Version 2.2.x

The ECAS Client PHP release 2.2 will no longer support PHP versions:

- 5.6
- 7.0

It will enforce compatibility as of version 7.1

PHP Version	PHP Active Support Till	Security updates till	ECAS PHP Client support
5.6	19-01-2017	31-12-2018	2.1.x
7.0	03-12-2017	03-12-2018	2.1.x
7.1	01-12-2018	01-12-2019	2.2
7.2	30-11-2019	30-11-2020	2.2

This version still uses Composer to manage its dependencies. The libraries though will no longer be packaged with the zip file and will need to be downloaded by the integrator.

Version 2.1

This is a BREAKING release.

Version 2.1 is a **breaking release**, thus tread carefully in upgrading your client.

- All classes are considered to be auto-loaded, no `require` or `require_once` statements are included in the SDK
- The structure of the SDK has been adapted to adhere to its namespace conventions. A class which has the namespace `eu\europa\ec\ecas\Cache`, should also reside in the folder `/eu/europa/ec/ecas/cache`.
- A new builder has been added for the `ServiceResponse`. `ServiceResponse` will also implement the `UserDetails` Interface.

A new feature has been added, called **Interceptors**. Interceptors can be chained and will be executed after the validation of the service ticket. The goal of the Interceptors is to enrich the `UserDetails/ServiceResponse` with data from a local system.

The last release also contained a small "issue": for testing purposed, CURL did not perform a `SSL_VERIFYPEER` or `SSL_VERIFYHOST`. This has been reverted an the validation is done successfully.

Version 2.0

The "old" PHP client's time had come, and needed to be upgraded to the more modern PHP standards. Also with [PHP7](#), we'll - finally - have scalar type and return type declarations, anonymous classes, etc. Also, the "old" PHP client lacked structured classes that might improve reusability and a single code base.

The new version of the ECAS PHP Client is currently supporting PHP5.6 and upwards.

Composer

The new client uses [Composer](#) to manage local dependencies. Composer will not be required for installing the ECAS PHP Client, as it will be shipped with all necessary libraries within the distribution package.

Installation

Composer

Check Composer connection

```
# Make sure to configure the proxies first
$ export https_proxy='http://<USER>:<PWD>@158.169.9.13:8012'
$ export http_proxy='http://<USER>:<PWD>@158.169.9.13:8012'

# See if you can use Composer
$ composer diagnose
Checking composer.json: WARNING
No license specified, it is recommended to do so. For closed-source
software you may use "proprietary" as license.
require.monolog/monolog : exact version constraints (1.24.0) should be
avoided if the package follows semantic versioning
Checking platform settings: OK
Checking git settings: OK
Checking http connectivity to packagist: OK
Checking https connectivity to packagist: OK
Checking HTTP proxy: OK
Checking HTTP proxy support for request_fulluri: OK
Checking HTTPS proxy support for request_fulluri: OK
Checking github.com rate limit: OK
Checking disk free space: OK
Checking pubkeys:
Tags Public Key Fingerprint: 57815BA2 7E54DC31 7ECC7CC5 573090D0 87719BA6
8F3BB723 4E5D42D0 84A14642
Dev Public Key Fingerprint: 4AC45767 E5EC2265 2F0C1167 CBBB8A2B 0C708369
153E328C AD90147D AFE50952
OK
Checking composer version: OK
Composer version: 1.8.4
PHP version: 7.1.23
PHP binary path: /usr/bin/php
```

Install the dependencies

The install should be invoked in the folder of the project. Dependencies will be installed in the /vendor folder.

```
$ composer install
Loading composer repositories with package information
Installing dependencies (including require-dev) from lock file
Warning: The lock file is not up to date with the latest changes in
composer.json. You may be getting outdated dependencies. Run update to
update them.
Package operations: 14 installs, 0 updates, 0 removals
  - Installing psr/log (1.1.0): Downloading (100%)
  - Installing symfony/polyfill-ctype (v1.10.0): Downloading
(100%)
  - Installing symfony/filesystem (v4.2.3): Downloading (100%)
  - Installing symfony/yaml (v4.2.3): Downloading (100%)
  - Installing symfony/serializer (v4.2.3): Downloading (100%)
  - Installing symfony/options-resolver (v4.2.3): Downloading
(100%)
  - Installing symfony/config (v4.2.3): Downloading (100%)
  - Installing monolog/monolog (1.24.0): Downloading (100%)
  - Installing theorchard/monolog-cascade (0.5.1): Downloading
(100%)
  - Installing symfony/contracts (v1.0.2): Downloading (100%)
  - Installing psr/container (1.0.0): Downloading (100%)
  - Installing symfony/dependency-injection (v4.2.3): Downloading
(100%)
  - Installing pdepend/pdepend (2.5.2): Downloading (100%)
  - Installing phpmd/phpmd (2.6.0): Downloading (100%)
# ... bla bla bla
Generating autoload files
```

How does the PHP Client work?

No better way to explain how it works than by an example:

ECAS Client sample

```
<?php
require_once __DIR__ . '
/autoload.php';
use eu\europa\ec\ecas\Client;
use
eu\europa\ec\ecas\utils\StringUtils;

$ecas_client = new Client();

$page_restricted =
$ecas_client->isPageAccessRestricted();
$user_authenticated =
$ecas_client->hasUserBeenAuthenticated();
$ticket = isset($_GET
['ticket']) ? $_GET
['ticket'] : null;

if ($page_restricted
    &&
    !$user_authenticated
    && StringUtils::
isEmpty($ticket)) {
    $ecas_client->requestECASAuthentication();
} else {
    if (!$ecas_client->hasUserBeenAuthenticated()
        && !StringUtils::
isEmpty($ticket)) {
        try {
            $user =
$ecas_client->validateServiceTicket
($ticket, true);
            $ecas_client->setAuthenticatedUser($user);
        } catch (\Exception
$e) {

// handle error
            trigger_error($e->getMessage(), E_USER_ERROR);
        }
    }
}
```

Line N°	
2	The SDK assumes autoloading. Have a look at the autoload file which is included on how to handle that.
3	We are using namespaces in PHP, so include the namespace for the ECAS Client
6	Create a new ECAS Client instances. The constructor has a default parameter for the Properties interface. More about this later.
8	Check if the current page is in the restricted url mapping
9	Check if the user has been authenticated already
10	Get the ticket parameter from the query parameters
12-13	if the page is restricted, and the user is not authenticated, request to be authenticated by ECAS. The service parameter will be generated using the information of the current visited url.
15	if the user hasn't been authenticated, but you have a ticket, try to validate it.
17	Validate the Service Ticket. If the second parameter is provided and set to true a new Proxy Granting Ticket will be requested. If the validation fails, it will trigger an exception.
18	Set the authenticated user.

Configuration

When you create a new instance of the ECAS Client, you can pass as an optional argument a instance of Properties.

By default, it will take the JsonProperties implementation, which uses a file called config.json (in /sdk/Properties).

ECAS Client config.json

```
{
  "baseUrl" : "https://localhost:7002",
  "baseService": "https://localhost",
  "validationUrl" : "/cas/strictValidate",
  "pgtCallbackUrl" : "https://ecas.localhost/proxyCallback.php",
  "requestingUserDetails" : true,
  "groupFilters" : ["*"],
  "acceptedStrengths" : [],
  "acceptedTicketTypes" : ["SERVICE", "PROXY"],
  "restrictedUrlMappings": [
    "/restricted.php"
  ],
  "certificatePath" : "../certificates/localhost.pem",
  "interceptors": [
    "demo\\DummyInterceptor1",
    "demo\\DummyInterceptor2"
  ],
  "cacheEngine" : {
    "type" : "APC"
  }
}
```

Some explanation might be required:

Key	Explanation
baseUrl	The ECAS url you want to use, eg. ■ https://localhost:7002, or ■ https://ecas.acceptance.ec.europa.eu, or ■ https://ecas.ec.europa.eu ■ ...

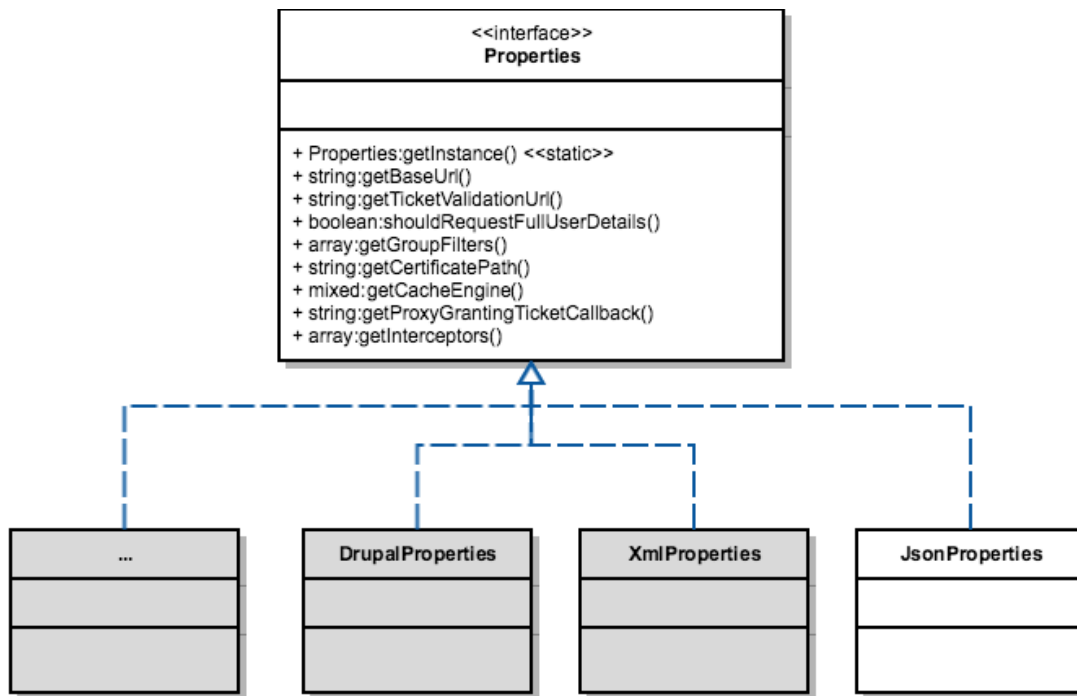
baseService	Optional The SDK will use this value only if it is provided in the configuration. If the value is not set (null), it will compose the baseService from the servers configuration: <pre> \$_protocol = ((array_key_exists('HTTPS', \$_SERVER) && \$_SERVER ['HTTPS'] == 'on') ? 'https' : 'http'); \$_host = \$_SERVER ['HTTP_HOST']; \$_baseService = sprintf("% s://%s%s", \$_protocol, \$_host, \$this->composeCleanRequestUrl (\$_request_uri)); </pre>
validationUrl	Defines which validation url is used for the validation of service tickets, must be one of the following: ▪ /cas/strictValidate: reject self-registered users, sponsored accounts and inter-institutional users ▪ /cas/interinstitutionalValidate: reject self-registered users and sponsored accounts ▪ /cas/sponsorValidate: reject self-registered users; ▪ /cas/laxValidate: accept any user
pgtCallbackUrl	The Proxy Ticket Granting callback url. This url needs to be reachable by the ECAS instances that you defined in the baseUrl property. More about the PGT flow can be found later in this document.
requestingUserDetails	A boolean indicating that the SDK Client should request full user details when validating the service ticket of the authenticated user.
groupFilters	An array of groups that need to be returned. You can use "*" to return all, or a patten to match groups e.g. "DIGIT*"
acceptedStrengths	The strengths that need to be enabled when displaying the ECAS landing page
acceptedTicketTypes	The list of acceptedTicketTypes that can be used to validated a ticket. Defaulted to SERVICE and PROXY, it can be extended to use DESKTOP as type too.
restrictedUrlMappings	This array is used to map to urls that require the user to be authenticated. These are matched using regular expressions.

certificatePath	The (absolute) path to the trusted certificate file that needs to be used to interact with ECAS (service validation). This file should be a text file with concatenated certificates in the Base64 encoded PEM format, such as used by the <i>curl</i> tool. If <i>baseUrl</i> above is an internal URL of the ECAS server (.cc.cec.eu.int) this file must contain the <i>CommisSign - 2</i> and <i>European Commission Root CA - 2</i> certificates. These can be downloaded from this page. If <i>baseUrl</i> is a Reverse Proxy URL of the ECAS server (e.g. .ec.europa.eu), then no special certificate file is needed.
interceptors	A list of interceptor classes, included their namespaces, that will be invoked after the Service Ticket Validation. They are chained, thus the value is passed from one interceptor to another, i.e. Service Ticket Validation Interceptor 1 Interceptor 2 ... Interceptor N
cacheEngine	Currently only supports APC, APCu and Memcached. Future implementations might bring other caching mechanisms. Note: a NullCacheEngine is also available, but it should NEVER be used in production environments. It is a simple implementation that stores the values in a static property.

Properties is an interface which aims at allowing multiple ways of implementing property management in the ECAS SDK.

ECAS SDK Properties Interface
<pre> <?php namespace eu\europa\ec\ecas\properties; interface Properties { public static function getInstance(); public function getBaseUrl(); public function getTicketValidationUrl(); public function shouldRequestFullUserDetails(); public function getGroupFilters(); public function getAcceptedStrengths(); public function getAcceptedTicketTypes(); public function getRestrictedMappings(); public function getCertificatePath(); public function getCacheEngine(); public function getProxyGrantingTicketCallback(); public function getInterceptors(); } </pre>

Currently, only the `JsonProperties` implementation exists, but future implementations might include compatible `DrupalProperties`, `XmlProperties`, etc.



Certificates

When you access EU Login via the PHP Client, we advise you to add a certificate in the configuration to verify the authenticity of the server.

To access EU Login, you can use two types of URL:

- An external URL, which has a Root CA that it trusted, e.g. <https://ecas.ec.europa.eu>
- An internal URL, which is a self-signed certificate, e.g. <https://ecas.cc.cec.eu.int:7002>

The first case, accessing EU Login via the Reversed Proxy, will always work if you have an updated list of trusted CA's. The internal URL on the other hand will not work out of the box because the client has the following options set

CURL SSL settings

```

// set to 1/true to verify the peer's certificate
curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, true);

// 2 means we'll make sure it matches the provided hostname in
the SSL handshake
curl_setopt($ch, CURLOPT_SSL_VERIFYHOST, 2);
  
```

If you try to access the internal URL, you get a CURL error in the log, such as:

```

2018-12-21 07:22:34 EULogin_Client [WARNING] - [Request Ticket Validation]
- Received invalid response: Context: {"httpCode":0,"errorNumber":60,"
error":"SSL certificate problem: unable to get local issuer
certificate"} Extra: {...}
  
```

To resolve this issue, follow these steps:

- From [Downloads-Certificates](#), download
 - CommisSign - 2.cer (SHA-2)
 - European Commission Root CA - 2.cer (SHA-2)
- Retrieve the certificate from the server:

```
$ echo | openssl s_client -connect ecas.cc.cec.eu.int:7002 2>&1 | sed
-ne '/-BEGIN CERTIFICATE-/,/-END CERTIFICATE-/p' > cert.pem
```

- Combine the certificates into a single file:

```
$ cat EuropeanCommission2.cer > ecas-prod.cer
$ cat CommisSign2.cer >> ecas-prod.cer
$ cat cert.pem >> ecas-prod.cer
```

- Verify the certificate

```
$ openssl verify ecas-prod.cer
ecas-prod.cer: CN = European Commission Root CA - 2
error 18 at 0 depth lookup:self signed certificate
OK
```

- Configure the PHP Client to use the generated [ecas-prod.cer](#) file.

CacheEngine

Your properties file will define a "cacheEngine" entry, which will return an object used for initialising the Caching mechanisms. The current implementation will only allow APC and Memcached as cache engine, and is used to temporarily store the PGT IOU and PGTID mapping.

As with the Properties interface, the idea behind the CacheEngine interface is that it allows to implement multiple caching mechanisms, such as APC, memcached, custom caching etc, depending on the needs of your project.

ECAS PHP Client Cache interface

```
<?php

namespace eu\europa\ec\ecas\cache;
require_once __DIR__ . '/CacheElement.php';

interface Cache
{
    /**
     * Retrieves an element from the cache
     *
     * @param $key
     * @return CacheElement
     */
    public function get($key);

    /**
     * Stores the element in the cache
     *
     * @param CacheElement $element
     */
    public function set(CacheElement $element);

    /**
     * Removes an element from the cache
     *
     * @param CacheElement $element
     * @return bool
     */
    public function delete(CacheElement $element);

    /**
     * Checks if the key exists in the cache
     *
     * @param $key
     * @return bool
     */
    public function has($key);
}
```

NullCache

About NullCache

Note: a NullCacheEngine is also available, but it should NEVER be used in production environments. It is a simple implementation that stores the values in a static property.

Enable cache engine in properties

```
"cacheEngine" : {  
    "type" : "NULL"  
}
```

ApcCache

About APC

Dixit Wikipedia:

Alternative PHP Cache is a free and open (PHP license) framework that caches the output of the PHP bytecode compiler in shared memory, thus reducing parsing and disk I/O overhead for later requests; and a shared memory cache for user data. For an application consisting of a large source code base such as Drupal, a 3x increase in page generation speed is possible as a result.

...

APC was originally scheduled for inclusion into the PHP core no later than PHP 6. While multiple accelerator projects were considered desirable,[4] the focus has since moved to Optimizer Plus, and, later, Zend Opcache that is included in the core distribution as of PHP 5.5. Since March 2013, a beta version of APCu (APC User Cache) is available, in which all opcode caching abilities have been removed to support user caches in shared memory using the familiar APC API.

This cache is also the reason why the new version of the ECAS PHP Client will enforce a minimum PHP of version 5.6.

When should you not consider the ApcCache implementation:

- If you haven't got this pecl installed;
- If you have a php application that needs to share its cache across multiple servers

Enable cache engine in properties

```
"cacheEngine" : {  
    "type" : "APC"  
}
```

APCu Cache

About APCu

APCu is APC stripped of opcode caching.

The first APCu codebase was versioned 4.0.0, it was forked from the head of the APC master branch at the time.

PHP 7 support is available as of APCu 5.0.0.

APCu can provide a compatibility mode, such that it can provide a drop in replacement for the applicable parts of APC.

Enable cache engine in properties

```
"cacheEngine" : {  
  "type" : "APCU"  
}
```

Memcached

About Memcached

Dixit Wikipedia

Memcached (pronunciation: mem-cash-dee, mem-cashed) is a general-purpose distributed memory caching system. It is often used to speed up dynamic database-driven websites by caching data and objects in RAM to reduce the number of times an external data source (such as a database or API) must be read. Memcached is free and open-source software, licensed under the Revised BSD license.[2] Memcached runs on Unix-like operating systems (at least Linux and OS X) and on Microsoft Windows. It depends on the libevent library.

Memcached's APIs provide a very large hash table distributed across multiple machines. When the table is full, subsequent inserts cause older data to be purged in least recently used (LRU) order.[3][4] Applications using Memcached typically layer requests and additions into RAM before falling back on a slower backing store, such as a database.

Memcached has no internal mechanism to track misses which may happen. There are some 3rd party applications such as mctop which can be used or if your operating system supports Dtrace you can use dtrace script.

Enabled cache engine in properties

```
"cacheEngine" : {  
  "type" : "MEMCACHED",  
  "server": {  
    "host" : "localhost",  
    "port" : 32768  
  }  
}
```

Logging

The application currently implements a LoggingFactory, which will return a Logger, compliant to the [Logging PSR-3](#) interface. We have implemented Monolog at the moment, configurable via the monolog.yaml configuration file.

Configuration

```

formatters:
  dashed:
    class: Monolog\Formatter\LineFormatter
    format: "%datetime% %channel% [%level_name%] - %message% \t
Context: %context% \t Extra: %extra%\n"
handlers:
  console:
    class: Monolog\Handler\StreamHandler
    level: DEBUG
    formatter: dashed
    processors: [memory_processor]
    stream: php://stdout
  info_file_handler:
    class: Monolog\Handler\StreamHandler
    level: DEBUG
    formatter: dashed
    stream: ./eulogin-phpclient-log.log
processors:
  web_processor:
    class: Monolog\Processor\WebProcessor
  memory_processor:
    class: Monolog\Processor\MemoryUsageProcessor
loggers:
  EULogin_Client:
    handlers: [console, info_file_handler]
    processors: [web_processor]
  EULogin_MemCached:
    handlers: [console]
    processors: [web_processor]
  EULogin_JsonProperties:
    handlers: [console]
    processors: [web_processor]
  EULogin_AutoLoad:
    handlers: [console]
    processors: [web_processor]

```

FUTURE VERSION: As with the Interceptors, we'll make the Logger configurable in the future. This would imply that it is injectable via the configuration.

How to use

An example on how to fetch the logger and to invoke a debug logging statement.

```

public function __construct()
{
    $this->_logger = LoggerFactory::getLogger('Name_The_Logger');
}

function chain(UserDetails $userDetails)
{
    ...
    $this->_logger->debug("Some debug information");
    ...
}

```

Interceptors

You don't need to call or chain the interceptors yourself. This will be done by the ECAS Client after the validation of the service ticket.

Interceptors
<pre> // validate service ticket and fetch UserDetails \$parsingResult = ServiceResponseParser::getInstance(\$this->_properties)- >parseTicketValidationRequest(\$result); \$_interceptors = \$this->_properties->getInterceptors(); if (sizeof(\$_interceptors) > 0) { // chain interceptors foreach (\$_interceptors as \$_interceptor) { \$this->_log->debug("Chaining interceptor", array("class" => get_class(\$_interceptor))); \$parsingResult = \$_interceptor->chain(\$parsingResult); } } </pre>

Interface

An Interceptor can be defined in the properties of the SDK, and needs to implement the `Interceptor-interface`

Interceptor interface

```
<?php
namespace eu\europa\ec\ecas\interceptor;

use eu\europa\ec\ecas\parser\UserDetails;

interface Interceptor
{
    /**
     * The interceptors are sequentially executed after the validation of
     the service ticket. It allows enriching the
     * UserDetails object with more information. The return object must
     comply to the UserDetails interface.
     *
     * @param UserDetails $userDetails
     * @return UserDetails
     */
    function chain(UserDetails $userDetails);
}
```

Implementation example

An example could be

Interceptor implementation example

```
<?php

// we have used the "demo" namespace deliberately here
namespace demo;

use eu\europa\ec\ecas\interceptor\Interceptor;
use eu\europa\ec\ecas\parser\UserDetails;

class DummyInterceptor1 implements Interceptor
{
    /**
     * DummyInterceptor1 constructor.
     */
    public function __construct()
    {
        // Do whatever is necessary to instantiate the interceptor. E.g.
        enable logging.
    }

    /**
     * The interceptors are sequentially executed after the validation of
     * the service ticket. It allows enriching the
     * UserDetails object with more information. The return object must
     * comply to the UserDetails interface.
     *
     * @param UserDetails $userDetails
     * @return UserDetails
     */
    function chain(UserDetails $userDetails)
    {
        // a new DummyInterceptorDetails class, extending the
        // ServiceResponse, adhering the UserDetails interface
        // is created, and received the previous UserDetails as a
        // constructor parameter.
        $_demoData = new DummyInterceptorDetails($userDetails);

        // the new object will have new getters/setters/methods.
        // But at the end, it is still an implementation of the UserDetails interface!
        $_demoData->setDummyUser('...');
        $_demoData->setDummyEmail('...');

        // return the extended object
        return $_demoData;
    }
}
```